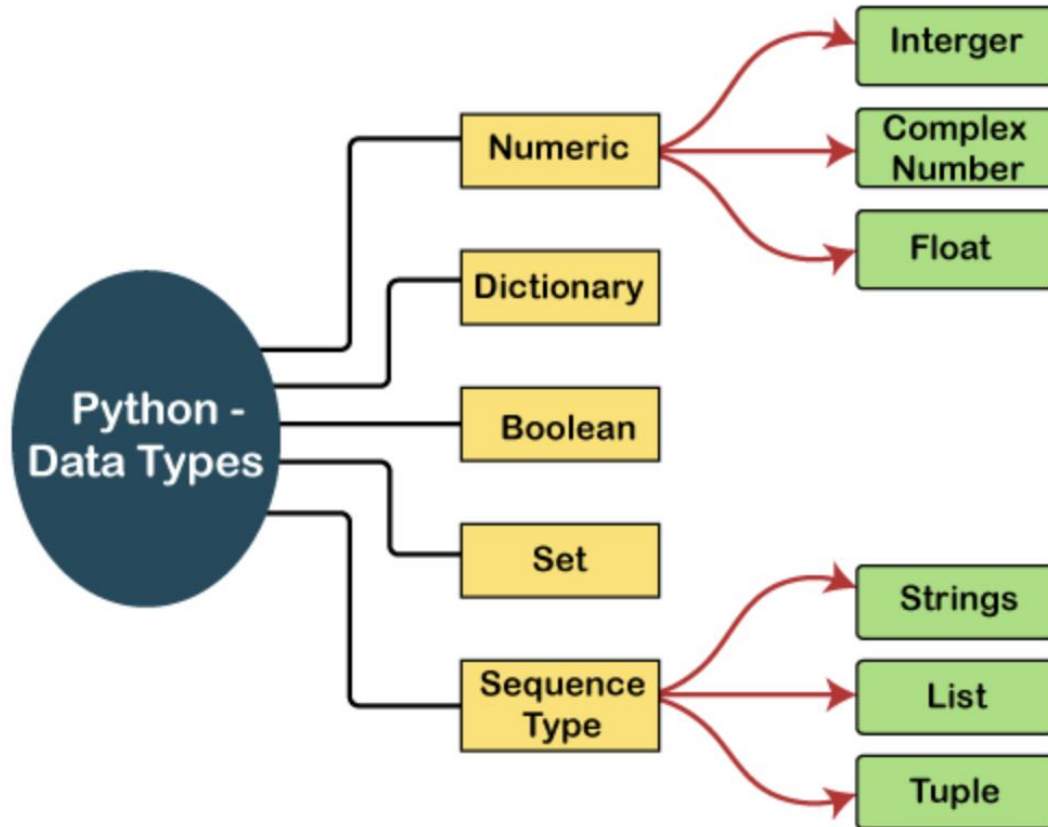




PYTHON STANDARD TYPES



Type Range

Sintassi di range()

In Python, the **range** type is an immutable sequence type that represents a lazily-evaluated arithmetic progression of integers. It is frequently used in **for loops** or in contexts that require efficient iteration over a sequence of integer values without allocating the entire list in memory.

The object supports efficient indexing, membership testing, and slicing, all implemented without materializing the full sequence.

```
range(stop)
```

```
# Produces an arithmetic sequence from 0 up to, but not including, stop.
```

```
# The step value defaults to 1.
```

```
range(start, stop)
```

```
# Produces an arithmetic sequence from start up to, but not including, stop.
```

```
# The step value defaults to 1.
```

```
range(start, stop, step)
```

```
# Produces an arithmetic sequence from start up to, but not including, stop,
```

```
# advancing by the specified step value (which may be positive or negative).
```

Caratteristiche di range

Range is an iterable object (not a list!).

range does not immediately create a list in memory; instead, it generates numbers on demand, making it very memory-efficient.

```
range(3)  
first request produces 0  
second request produces 1  
third request produces 2
```



STRING

PYTHON BASIC

Sequence types: Strings

- Created by simply enclosing characters in either single- or double-quotes.
- It's enough to simply assign the string to a variable.
- Strings are immutable.

```
mystring = "Hi, I'm a string!"
```

Strings

- Strings are subtypes of the sequence data type.
- Strings are written with either single or double quotes encasing a sequence of characters.

```
s1 = 'This is a string!'
s2 = "Python is so awesome."
```

- As a subtype of the sequence data type, strings can be accessed element-wise as they are technically just sequences of character elements.
- We can index with typical bracket notation, as well as

```
>>> s1 = "This is a string!"
>>> s2 = "Python is so awesome."
>>> print s1[3]
s
>>> print s2[5:15]
n is so aw
```

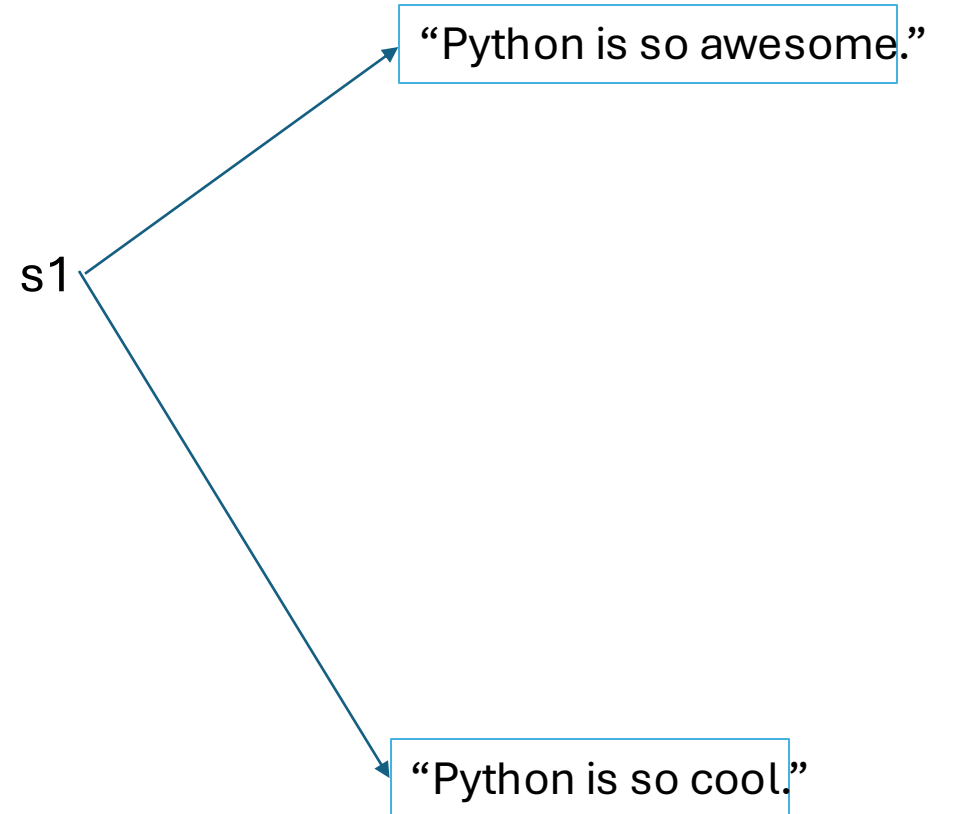
Note that there is no character data type in Python. A character is simply represented as a string with one character.

Operator	Description
+	concatenation operator used to join the strings given either side of the operator.
*	repetition operator. It concatenates the multiple copies of the same string.
[]	Islice operator. It is used to access the sub-strings of a particular string.
[:]	range slice operator. It is used to access the characters from the specified range.
in	membership operator. It returns if a particular sub-string is present in the specified string.
not in	membership operator and does the exact reverse of in. It returns true if a particular substring is not present in the specified string.
r/R	It is used to specify the raw string. Raw strings are used in the cases where we need to print the actual meaning of escape characters such as "C://python". To define any string as a raw string, the character r or R is followed by the string.
%	It is used to perform string formatting. It makes use of the format specifiers used in C programming like %d or %f to map their values in python. We will discuss how formatting is done in python.

Modifying strings

- Strings are *immutable* – you cannot update the value of an existing string object. However, you can reassign your variable name to a new string object to perform an “update”.

```
>>> s1 = "Python is so awesome."  
>>> s1 = "Python is so cool."
```



No modify

```
s1 = 'This is a string!'
print(s1[0]) = "a"
```

```
<module> ----> 1 Str[0]="a"
```

```
TypeError: 'str' object does not support item assignment
```

Modifying strings

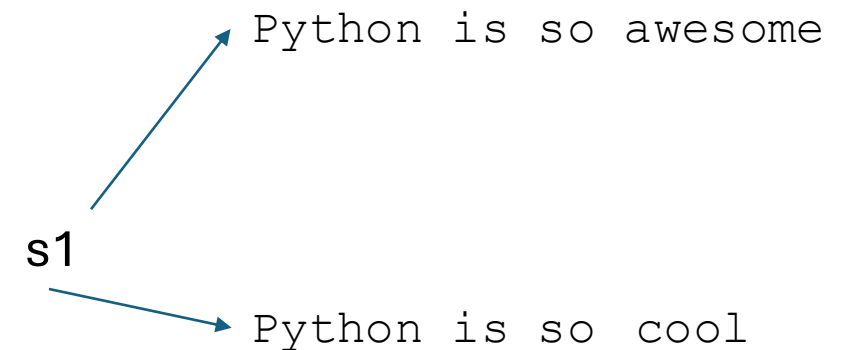
Alternatively, we could have done the following:

Python is so cool.

This will create a substring “Python is so ”, which is concatenated with “cool.”, stored in memory and associated with the name s1.

The “+” operator can be used with two string objects to concatenate them together.

```
>>> s1 = "Python is so awesome."  
>>> s1 = s1[:13] + "cool."
```



Modifying strings

```
>>> s1 = "Python"  
>>> s1 = s1 * 2  
>>> print(s1)
```

Alternatively, we could have done the following:

```
PythonPython
```

The “*” operator can be used to concatenate multiple copies of a single string object.

We also have `in` and `not in` available for testing character membership within a string.

```
>>> s1 = "Python is so awesome."  
>>> s1 = s1[:13] + "cool."
```

```
x = "pippo"
```

```
print("pi" in x)
```

```
x = "pippo"
```

```
print("pi" in x)
```

Sequence types: Strings

- Python supports a number of escape sequences such as `'\t'`, `'\n'`, etc.
- Placing `'r'` before a string will yield its raw value.
- There is a string formatting operator `'%'` similar to C.
- Two string literals beside one another are automatically concatenated together.

```
print ("\tHello,\n")
print ("\tWorld!\n")
print ("Python is " "so cool.")
```

Built-in string methods

- Python includes a number of built-in string methods that are incredibly useful for string manipulation. Note that these *return* the modified string value; we cannot change the string's value in place because they're immutable!
- `s.upper()` and `s.lower()`

```
>>> s1 = "Python is so awesome."  
>>> print (s1.upper())  
PYTHON IS SO AWESOME.  
>>> print (s1.lower() )  
python is so awesome.
```

Built-in string methods

- `s.isalpha()`, `s.isdigit()`, `s.isalnum()`, `s.isspace()` – return True if string `s` is composed of alphabetic characters, digits, either alphabetic and/or digits, and entirely whitespace characters, respectively.
- `s.islower()`, `s.isupper()` – return True if string `s` is all lowercase and all uppercase, respectively.

```
>>> "WHOA".isupper()
True
>>> "12345".isdigit()
True
>>> "\n".isspace()
True
>>> "hello!".isalpha()
False
```

Built-in string methods

- `str.split([sep[, maxsplit]])` – Split *str* into a list of substrings. The *sep* argument indicates the delimiting string (defaults to consecutive whitespace). The *maxsplit* argument indicates the maximum number of splits to be done (default is -1).
- `str.rsplit([sep[, maxsplit]])` – Split *str* into a list of substrings, starting from the right.
- `str.strip([chars])` – Return a copy of the string *str* with leading and trailing characters removed. The *chars* string specifies the set of characters to remove (default is whitespace).
- `str.rstrip([chars])` – Return a copy of the string *str* with only trailing characters removed.

Built-in string methods

```
>>> "Python programming is fun!".split()  
['Python', 'programming', 'is', 'fun!']  
>>> "555-867-5309".split('-')  
['555', '867', '5309']  
>>> "***Python programming is fun***".strip('*')  
'Python programming is fun'
```

Built-in string methods

- `str.capitalize()` – returns a copy of the string with the first character capitalized and the rest lowercase.
- `str.center(width[, fillchar])` – centers the contents of the string `str` in field-size `width`, padded by `fillchar` (defaults to a blank space). See also `str.ljust()` and `str.rjust()`.
- `str.count(sub[, start[, end]])` – return the number of non-overlapping occurrences of substring `sub` in the range `[start, end]`. Can use slice notation here.
- `str.endswith(suffix[, start[, end]])` – return True if the string `str` ends with suffix, otherwise return False. Optionally, specify a substring to test. See also `str.startswith()`.

Built-in string methods

```
>>> "i LoVe pYtHoN".capitalize()
'I love python'
>>> "centered".center(20, '*')
'*****centered*****'
>>> "mississippi".count("iss")
2
>>> "mississippi".count("iss", 4, -1)
1
>>> "mississippi".endswith("ssi")
False
>>> "mississippi".endswith("ssi", 0, 8)
True
```

Built-in string methods

- `str.find(sub[, start[, end]])` – return the lowest index in the string where substring *sub* is found, such that *sub* is contained in the slice `str[start:end]`. Return -1 if *sub* is not found. See also `str.rfind()`.
- `str.index(sub[, start[, end]])` – identical to `find()`, but raises a *ValueError* exception when substring *sub* is not found. See also `str.rindex()`.
- `str.join(iterable)` – return a string that is the result of concatenating all of the elements of *iterable*. The *str* object here is the delimiter between the concatenated elements.
- `str.replace(old, new[, count])` – return a copy of the string *str* where all instances of the substring *old* are replaced by the string *new* (up to *count* number of times).

Built-in string methods

```
>>> "whenever".find("never")
3
>>> "whenever".find("what")
-1
>>> "whenever".index("what")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: substring not found
>>> "-".join(['555', '867', '5309'])
'555-867-5309'
>>> " ".join(['Python', 'is', 'awesome'])
'Python is awesome'
>>> "whenever".replace("ever", "ce")
'whence'
```

The string module

- .All of these built-in string methods are methods of any string object. They do not require importing any module or anything – they are part of the core of the language.
- There is a string module, however, which provides some additional useful string tools. It defines useful string constants, the string formatting class, and some deprecated string functions which have mostly been converted to methods of string objects.

String constants

```
>>> import string
>>> string.ascii_letters
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'
>>> string.ascii_lowercase
'abcdefghijklmnopqrstuvwxyz'
>>> string.ascii_uppercase
'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
>>> string.digits
'0123456789'
>>> string.hexdigits
'0123456789abcdefABCDEF'
```

String constants

```
>>> import string
>>> string.ascii_lowercase #locale-dependent
'abcdefghijklmnopqrstuvwxyz'
>>> string.ascii_uppercase #locale-dependent
'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
>>> string.ascii_letters # lowercase+uppercase
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'
>>> string.digits
'0123456789'
>>> print string.punctuation
!"#$%&'()*+,-./:;<=>?@[\\]^_`
```


String constants

- `string.whitespace` – a string containing all characters that are considered whitespace. On most systems this includes the characters space, tab, linefeed, return, formfeed, and vertical tab.
- `string.printable` – string of characters which are considered printable. This is a combination of digits, letters, punctuation, and whitespace.

class string.Formatter

```
format(*args, **kwargs)
```

```
vformat(format_string, args, kwargs)
```

format takes a format string and an arbitrary set of positional and keyword arguments. It is just a wrapper that calls vformat().

vFormat does the actual work of formatting. It is exposed as a separate function for cases where you want to pass in a predefined dictionary of arguments, rather than unpacking and repacking the dictionary as individual arguments using the *args and **kwargs syntax. vformat() does the work of breaking up the format string into character data and replacement fields. It calls the various methods described below.

The string on which this method is called can contain literal text or replacement fields delimited by braces {}. Each replacement field contains either the numeric index of a positional argument, or the name of a keyword argument. A copy of the string is returned where each replacement field is replaced with the string value of the corresponding argument

String formatting

```
>>> '{0}, {1}, {2}'.format('a', 'b', 'c')
'a, b, c'
>>> '{} , {} , {}'.format('a', 'b', 'c')
'a, b, c'
>>> '{2}, {1}, {0}'.format('a', 'b', 'c')
'c, b, a'
>>> '{2}, {1}, {0}'.format(*'abc')
'c, b, a'
>>> '{0}{1}{0}'.format('abra', 'cad')
'abracadabra'
```

String formatting

You can also use keyword arguments to the format function to specify the value for replacement fields

```
>>> 'Coords: {lat}, {long}'.format(lat='37.24N', long='-115.81W')
'Coords: 37.24N, -115.81W'
>>> coord = {'lat': '37.24N', 'long': '-115.81W'}
>>> 'Coords: {lat}, {long}'.format(**coord)
'Coords: 37.24N, -115.81W'
```

String formatting

Within the replacement field, you are able to access attributes and methods of the object passed as an argument to format. Here, we pass a complex number as an argument, but we access its member attributes in the replacement field.

```
>>> c = 2+3j
>>> '{0} has real part {0.real} and imaginary part {0.imag}.'.format(c)
'(2+3j) has real part 2.0 and imaginary part 3.0.'
```

```
>>> coord = (3, 5)
>>> 'X: {0[0]}; Y: {0[1]}'.format(coord)
'X: 3; Y: 5'
```

String formatting

- There are reserved sequences for specifying justification and alignment within a replacement field.

```
>>> '{:<30}'.format('left aligned')
'left aligned'
>>> '{:>30}'.format('right aligned')
'
                right aligned'
>>> '{:^30}'.format('centered')
'
                centered
>>> '{:*^30}'.format('centered') # use '*' as a fill char
'*****centered*****'
```

String formatting

- There are a number of options for formatting floating-point numbers.

```
>>> '{:+f}; {:+f}'.format(3.14, -3.14) # show sign always
'+3.140000; -3.140000'
>>> '{: f}; {: f}'.format(3.14, -3.14) # show space for positive
' 3.140000; -3.140000'
>>> '{:-f}; {:-f}'.format(3.14, -3.14) # show only minus
'3.140000; -3.140000'
>>> '{:.3f}'.format(3.14159) # limit to three dec places
'3.142'
```

Sequence Types: Unicode Strings

- Unicode strings can be used to store and manipulate Unicode data.
- As simple as creating a normal string (just put a 'u' on it!).
- Use Unicode-Escape encoding for special characters.
- Also has a raw mode, use 'ur' as a prefix.
- To translate to a regular string, use the .encode() method.
- To translate from a regular string to Unicode, use the unicode() function.

```
newunicode = u'\xe4\xfc'
print (newunicode)
newstr = newunicode.encode('utf-8')
print (newstr)
```

Output:

äü

b'\xc3\xa4\xc3\xb6'

Costant none

- In Python, **None** is a special constant that represents the absence of a value or a null value. It is similar to null in other programming languages. It is often used to indicate that a variable has no value assigned to it, or to denote the end of a list or sequence.

Assignement

`x = None`

comparing

`x == None`

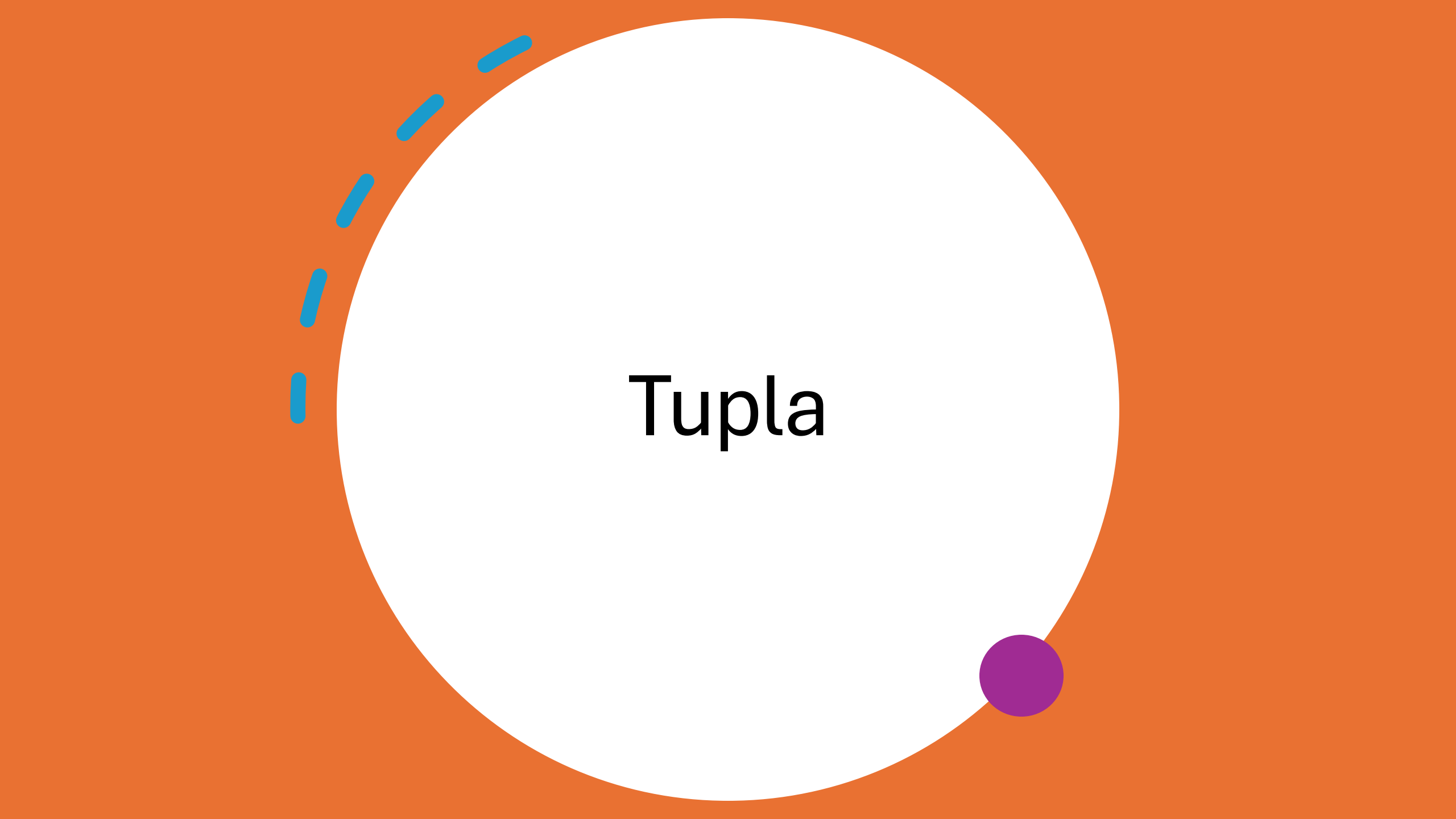
`x is None`

Function

Default parameter

Return value

```
if x is None:
    print("x is None")
else:
    #use of x
```



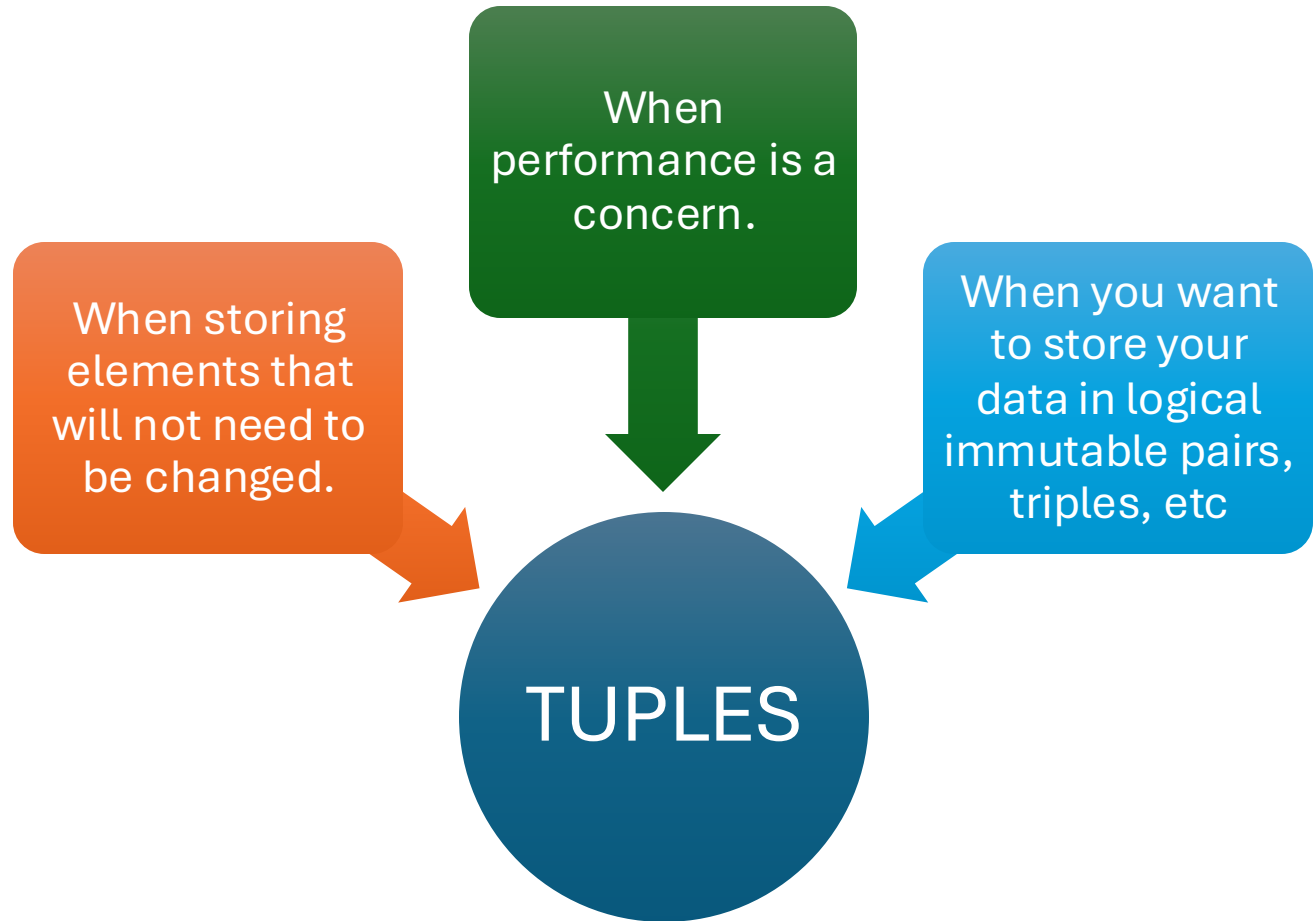
Tupla

Tipo tupla (tuple)

In Python, a tuple is an immutable sequence of elements of any type. Tuple characteristics:

- Ordered
- Indexable
- Immutable
- Can contain duplicates
- Concatenation
- Slicing
- Retrieving its length

When to use tuples



Tuple operations

- Tuples are very similar to lists and support a lot of the same operations.
- **Accessing elements**: use bracket notation (e.g. `t1[2]`) and slicing.
- Use `len(t1)` to obtain the length of a tuple.
- The universal immutable sequence type operations are all supported by tuples.
 - **+, ***
 - **in, not in**
 - **min(t), max(t), t.index(x), t.count(x)**

Constructing tuples

- An empty tuple can be created with an empty set of parentheses.
- Pass a sequence type object into the tuple() constructor.

```
>>> t3 = ()
```

```
>>> t1 = (1, 2, 3, 4)
```

- Tuples can be initialized by listing comma-separated values. These do not need to be in parentheses but they can be.
- One quirk: to initialize a tuple with a single value, use a trailing comma.

```
>>> t2 = "a", "b", "c", "d"
```

```
>>> t4 = ("red", )
```

Tupla indexing

```
persona = ("Vincenza", 35, "Catania")    # elements of different types
Tuplavuota = ()                          #no elements
T = (10,3,10)                            # duplicate elements
print(persona)
Print(type(persona))
```

(Vincenza,35,Catania
<class,tupla>

0	1	2
Vincenza	35	Catania
-3	-2	-1

```
print(persona[1])
```

35

```
print(persona[-1])
```

Catania

Operator + and *

```
persona = ("Vincenza", 35, "Catania")  
T = (10,3,10)  
nuovaT = persona + T  
print(nuovaT)  
print(type(nuovaT))
```

```
(Vincenza,35,Catania,10,3,10)  
<class,tupla>
```

```
T = (10,3,10)  
nuovaT2 = T * 3  
print(nuovaT2)  
print(type(nuovaT2))
```

```
(10,3,10,10,3,10,10,3,10)  
<class,tupla>
```

Function on tuple

Function	Description	Example	Output
<code>len()</code>	Returns the length of the tuple	<code>len((1, 2, 3))</code>	3
<code>max()</code>	Returns the largest element	<code>max((3, 7, 1))</code>	7
<code>min()</code>	Returns the smallest element	<code>min((3, 7, 1))</code>	1
<code>sum()</code>	Sums the numeric elements	<code>sum((1, 2, 3))</code>	6
<code>tuple()</code>	Converts another structure into a tuple	<code>tuple([1, 2])</code>	(1, 2)

Tuple Methods

Method	Description	Example	Output
<code>.count(x)</code>	Counts the number of occurrences of a value	<code>(1, 2, 2, 3).count(2)</code>	2
<code>.index(x)</code>	Returns the index of the first occurrence of a value	<code>(10, 20, 30).index(20)</code>	1

Tuple Slicing

```
numeri = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

print(numeri[2:6])      # Output: (2, 3, 4, 5)
print(numeri[:4])       # Output: (0, 1, 2, 3)
print(numeri[5:])       # Output: (5, 6, 7, 8, 9)
print(numeri[::2])      # Output: (0, 2, 4, 6, 8)
print(numeri[::-1])     # Output: (9, 8, 7, 6, 5, 4, 3, 2, 1, 0) (inversione)
```

Using for on tuple

```
numeri = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

Somma = 0
for x in numeri:
    Somma += x

print(Somma)
```

Immutability of a tuple

```
persona = ("Vincenza", 35, "Catania")  
Persona[0] = 'Giuseppe'      #ERROR
```

In operator

elemento in collezione

```
T = (3,4,'v', 'ciao')
```

```
If 'v' in T:
```

```
    print('v appartiene alla stringa')
```

```
# Starting string
testo = "questa è una semplice stringa senza simboli di
interpunzione"

# Working variables
parola = ""
tupla_parole = ()

# Loop through the string character by character
for c in testo:
    if c != " ":
        parola += c
    else:
        if parola != "":
            tupla_parole += (parola,)
            parola = ""

# Add the last word (if present)
if parola != "":
    tupla_parole += (parola,)

# Show the result
print(tupla_parole)
```

Tuple packing e unpacking

Tuple Packing

```
t = (1, 2, 3)
```

```
t = 1, 2, 3
```

Tuple Unpacking.

```
t = (1, 2, 3)  
a, b, c = t # unpacking  
print(a, b, c) # → 1 2 3
```

The number of variables must match the number of elements in the tuple, otherwise Python raises an error.

Unpacking parziale con “*”

```
t = (1, 2, 3, 4, 5)
a, *b, c = t
print(a) # 1
print(b) # [2, 3, 4]
print(c) # 5
```

```
x, y = 10, 20
x, y = y, x
print(x, y) # → 20 10
```

List vs. tuple



list: a compound, **mutable** data type that can hold items of varying types.

tuple: a compound, **immutable** data type that can hold items of varying types.



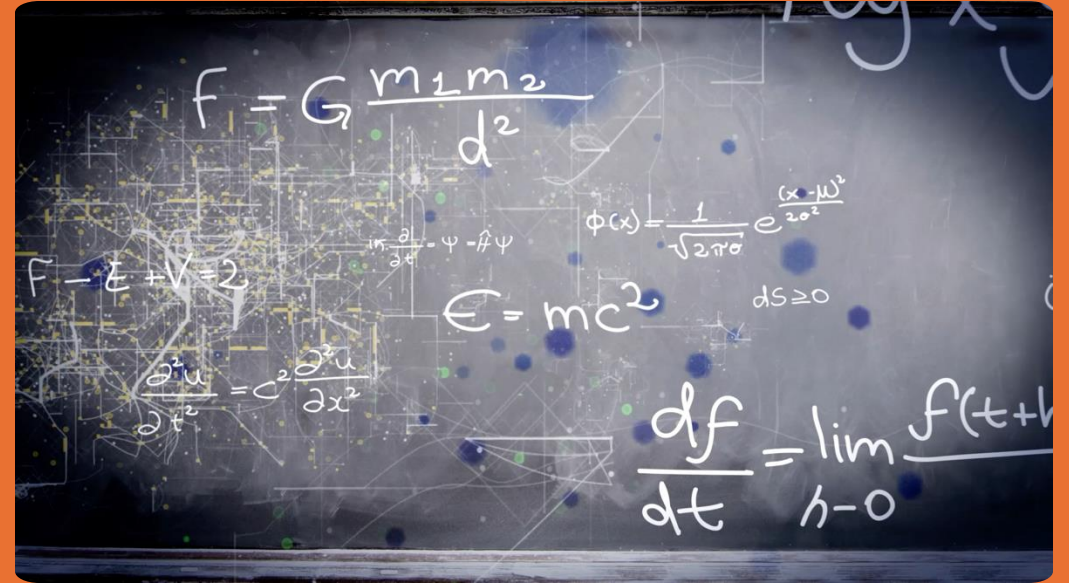


Tuple & Function returning

Tuple are extremely flexible and can be used both as function arguments and as return values.

By combining **packing** and **unpacking**, you can write very clean and elegant code.

FUNCTION



functions

A function is created with the `def` keyword. The statements in the block of the function must be indented.

```
def function_name(args):  
    statements
```

- `def` keyword is followed by the function name with round brackets enclosing the arguments and a colon. The indented statements form a body of the function.
- The `return` keyword is used to specify a list of values to be returned.

```
# Defining the function  
def print_greeting():  
    print ("Hello!")  
    print ("How are you today?")
```

```
print_greeting() # Calling the function
```

Hello!

How are you today?

If you write `return` without arguments the function returns **`None`** by default
If you don't write `return`, it's as if `return` were added at the end

return espressione
,

```
def nome_funzione():  
    return 10
```

```
def nome_funzione():  
    print('ciao')
```

return `

```
def nome_funzione():  
    print('ciao')  
    return
```

```
def nome_funzione():  
    return 10, 'ciao'
```

Parameter only by value

- All parameters in the Python language are passed by value, but each variable variables are just references to objects.
- However, only mutable objects can be changed in the called function.

```
def hello_func(name):  
    print ("Hello,", name, "!\n")  
    name = "Giuseppe"  
    return 1, 2  
  
myname = "Ben"  
a,b = hello_func(myname)  
print (myname)  
print (a, b)
```

Hello, Ben !

1 2

Functions

- What is the output of the following code?

Hello, Susan !

Hello, Peter !

Hello, William !

The names are now ['Susie', 'Pete', 'Will'] .

```
def hello_func(names):  
    for n in names:  
        print ("Hello,", n, "!" )  
    names = ("Francesco", "Giuseppe")  
    print(names)  
names = ('Susan', 'Peter', 'William')  
hello_func(names)  
print ("The names are now", names, ".")
```


A solution with functions

The Python interpreter will set some special environmental variables when it starts executing.

If the Python interpreter is running the module (the source file) as the main program, it sets the special `__name__` variable to have a value `"__main__"`.

This allows for flexibility in writing your modules.

Note: `__name__`, as with other built-ins, has two underscores on either side!

```
def even_fib():
    total = 0
    f1, f2 = 1, 2
    while f1 < 4000000:
        if f1 % 2 == 0:
            total = total + f1
        f1, f2 = f2, f1 + f2
    return total

if __name__ == "__main__":
    print(even_fib())
```

Positional /Default / Named Parameters

- Keyword arguments must follow positional arguments
- All keyword arguments must identify some parameter

Rules about Function Calls

- Keyword arguments must follow positional arguments
- All keyword arguments must identify some parameter
 - Even positional ones
- No parameter may receive a value more than once

Functions – Default parameter

- These calls can become a little cumbersome, especially if one of the arguments is likely to have the same value for every call.

Default argument values

- We can provide a default value for any number of arguments in a function.
- Allows functions to be called with a variable number of arguments.
- Arguments with default values must appear at the end of the arguments list!

```
def F(pos1, pos2, name1 = 1, name2 = "pippo", name3 = None):
```

Functions *positional arguments*

- Consider again our connecting function.

```
def connect(uname, pword, server = 'localhost', port = 9160):  
    # connecting code
```

- The following call utilizes *positional arguments*. That is, Python determines which formal parameter to bind the argument to based on its position in the list.

```
connect('admin', 'ilovecats', 'shell.cs.fsu.edu', 6379)
```

Functions: use argument name

- When the formal parameter is specified, this is known as a *keyword argument*.

```
connect (uname='admin', pword='ilovecats',  
        server='shell.cs.fsu.edu', port=6379)
```

- By using keyword arguments, we can explicitly tell Python to which formal parameter the argument should be bound. Keyword arguments are always of the form *kwarg = value*.
- If keyword arguments are used they must follow any positional arguments, although the relative order of keyword arguments is unimportant.

Positional and name parameter

- They can be mixed, but positional parameters must come first.
- In Python 3, you can use an asterisk (*) in the function definition to indicate that all parameters after that * must be passed by name.

```
def f(a, b, *, c, d):  
    print(a, b, c, d)  
  
f(1, 2, c=3, d=4) # ok  
# f(1, 2, 3, 4) # error
```

Functions: *param and ** PARAM

- Parameters of the form *param contain a variable number of arguments within a tuple.
- Parameters of the form **param contain a variable number of keyword arguments.

```
def connect(uname, *args, **kwargs):  
    # connecting code here
```

- This is known as *packing*.
- Within the function, we can treat args as a list of the positional arguments provided and kwargs as a dictionary of keyword arguments provided.

Functions

```
def connect(uname, *args, **kwargs):  
    print (uname)  
    for arg in args:  
        print (arg)  
    for key in kwargs.keys():  
        print (key, ":", kwargs[key])  
  
connect('admin', 'ilovecats', server='localhost', port=9160)
```

Output: ? admin
 ilovecats
 port : 9160
 server : localhost

functions

- We can use `*args` and `**kwargs` not only to define a function, but also to call a function. Let's say we have the following function.

```
def func(arg1, arg2, arg3):  
    print ("arg1:", arg1)  
    print ("arg2:", arg2)  
    print ("arg3:", arg3)
```

functions

- We can use `*args` to pass in a tuple as a single argument to our function. This tuple should contain the arguments in the order in which they are meant to be bound to the formal parameters.

```
>>> args = ("one", 2, 3)
>>> func(*args)
```

```
arg1: one
arg2: 2
arg3: 3
```

- We would say that we're *unpacking* a tuple of arguments here.

functions

- We can use `**kwargs` to pass in a dictionary as a single argument to our function. This dictionary contains the formal parameters as keywords, associated with their argument values. Note that these can appear in any order.

```
arg1: one  
arg2: 2  
arg3: 3
```

```
>>> kwargs = {"arg3": 3, "arg1": "one", "arg2": 2}  
>>> func(**kwargs)
```

Built_in function

- | | | | | |
|--------------|---------------|----------------|---------------|-------------|
| • abs() | ■ bytearray() | ■ locals() | ■ str() | ■ dict() |
| • all() | ■ eval() | ■ map() | ■ tuple() | ■ filter() |
| • bin() | ■ float() | ■ memoryview() | ■ type() | ■ hash() |
| • bool() | ■ format() | ■ object() | ■ vars() | ■ help() |
| • bytes() | ■ frozenset() | ■ open() | ■ zip() | ■ min() |
| • callable() | ■ getattr() | ■ ord() | ■ chr() | ■ set() |
| • compile() | ■ globals() | ■ pow() | ■ complex() | ■ hex() |
| • exec() | ■ hasattr() | ■ print() | ■ delattr() | ■ id() |
| • sum() | ■ iter() | ■ reversed() | ■ dir() | ■ setattr() |
| • any() | ■ len() | ■ range() | ■ divmod() | ■ slice() |
| • ascii() | ■ list() | ■ round() | ■ enumerate() | ■ sorted() |
| • input() | ■ int() | ■ isinstance() | ■ oct() | ■ next() |

FUNCTION OVERLOADING

- In Python, function overloading, where multiple functions can have the same name but different parameters, is not supported in the same way as in languages like C++ or Java.
- However, there are ways to simulate function overloading in Python, which includes:
 - Using Default Arguments
 - Using Variable-Length Arguments
 - Using Type Checking

```
def greet(name="Guest", greeting="Hello"):
    print(f"{greeting}, {name}!")
```

```
greet()          # Output: Hello, Guest!
greet("Alice")   # Output: Hello, Alice!
greet("Alice", "Goodbye") # Output: Goodbye, Alice!
```

```
def greet(name="Guest", greeting = None):
    print(f"{greeting}, {name}!")
```

```
greet()          # Output: Hello, Guest!
greet("Alice")   # Output: Hello, Alice!
greet("Alice", "Goodbye") # Output: Goodbye, Alice!
```

```
def greet(*args):  
    if len(args) == 1:  
        print(f"Hello, {args[0]}!")  
    elif len(args) == 2:  
        print(f"{args[1]}, {args[0]}!")  
    else:  
        print("Hello, Guest!")
```

```
greet("Alice")          # Output: Hello, Alice!  
greet("Alice", "Good evening") # Output: Good evening,  
Alice!
```

```
def add(a, b):  
    if isinstance(a, str) and isinstance(b, str):  
        return a + b # Concatenate strings  
    elif isinstance(a, int) and isinstance(b, int):  
        return a + b # Sum integers  
    else:  
        return "Invalid types"  
  
print(add(3, 4))      # Output: 7  
print(add("Hello", "World")) # Output: HelloWorld
```



```
def add(a, b):  
    print(2)
```

```
def add(a, b):  
    print(3)
```

```
print(a,b). #error
```

```
print(a,b,c) #ok
```

Structure of a .py file and code execution

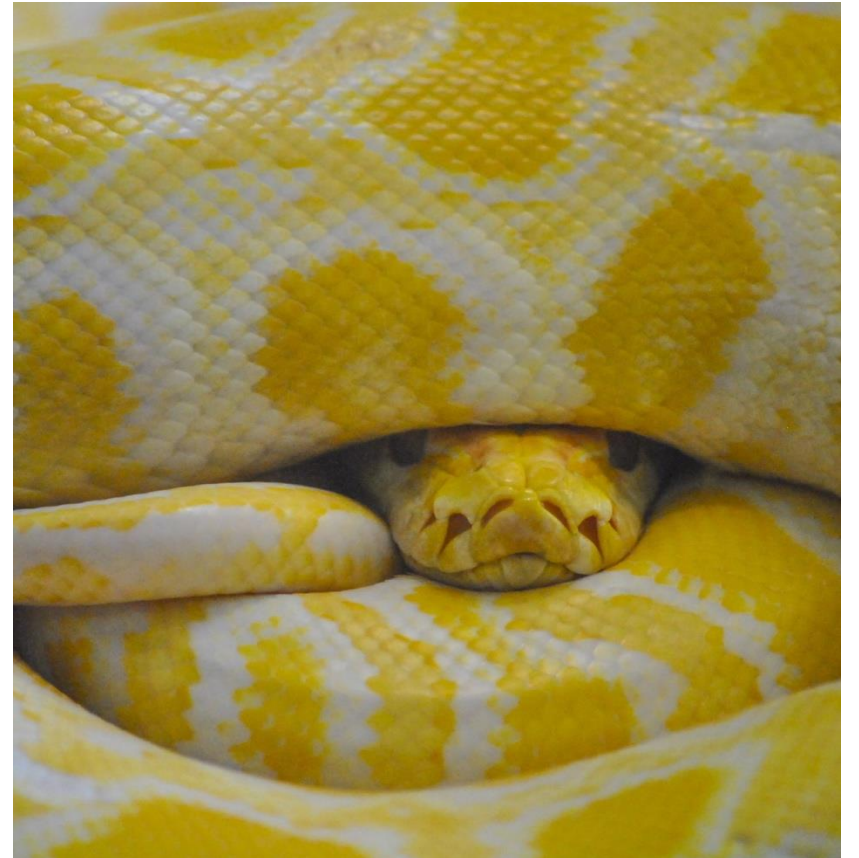
- A Python file is a simple text file with the **.py** extension. It can contain:
- **Imports** of modules or libraries
- **Function definitions**
- **Variables**, called global variables
- **Executable code** (that runs when the file is executed):
- **Special block `if __name__ == "__main__":`**
This is a way to say: run this code only if the file is executed directly, not if it is imported from another file.

```
if __name__ == "__main__":  
    funzione()
```



modules

- A module is a file containing Python definitions and statements.
- A document with definitions of functions and various statements written in Python is called a Python module.
- In Python, we can define a module in one of 3 ways:
 - Python itself allows for the creation of modules.
 - Similar to the re (regular expression) module, a module can be primarily written in C programming language and then dynamically inserted at run-time.
 - A built-in module, such as the itertools module, is inherently included in the interpreter.
- The file name is the module name with the suffix .py appended.



- Within a module, the module's name (as a string) is available as the value of the global variable `__name__`.
- If a module is executed directly however, the value of the global variable `__name__` will be `"__main__"`.
- Modules can contain executable statements aside from definitions. These are executed only the *first* time the module name is encountered in an import statement as well as if the file is executed as a script.

How a Python file is executed?

- When you run a Python file (for example with `python myfile.py`), the **Python interpreter**:
- **Reads the file** from top to bottom.
- **Executes the instructions** line by line.
- **Skips definitions** (functions or classes) until they are called.
- If it finds the block `if __name__ == "__main__":`, it executes it **only if the file is run directly**, not if it is imported from another file.

Example

```
# miofile.py

def saluta(nome):
    print(f"Ciao, {nome}!")

if __name__ == "__main__":
    saluta('Giuseppe')
```

Ciao Giuseppe

GLOBAL % LOCAL VARIABLE

```
# miofile.py
x = 10
def somma(y):
    return y + x

if __name__ == "__main__":
    z = somma(20)
    print(z)
```

Example: global variable

```
# miofile.py
X = 10
print(230)

def somma(y):
    return y + X

if __name__ == "__main__":
    z = somma(20)
    print(z)
```

230

30


```
def somma(a, b):  
    risultato = a + b  
    return risultato
```

```
x = somma(5, 3)  
print("La somma è:", x)
```

La somma è 8

```
def somma(a, b):  
    risultato = a + b  
    return risultato
```

```
a= int(input('Inserisci un intero'))  
b = int(input('Inserisci un intero'))
```

```
x = somma(a, b)  
print("La somma è:", x)
```

```
def leggi_numero():  
    return float(input("Inserisci un numero: "))
```

```
def calcola_media(a, b):  
    return (a + b) / 2
```

```
def main():  
    x = leggi_numero()  
    y = leggi_numero()  
    media = calcola_media(x, y)  
    print("La media è:", media)
```

```
main()
```

Scope e variable life time

```
def leggi_numero():  
    return float(input("Inserisci un numero: "))  
  
def calcola_media(a, b):  
    return (a + b) / 2  
  
x = leggi_numero()  
y = leggi_numero()  
media = calcola_media(x, y)  
print(a)  
print("La media è:", media)
```

Inserisci un numero 10

Inserisci un numero. 20

NameError: name 'a' is not defined

LOCAL, GLOBAL VARIABLE E LIFETIME

LOCAL

- A local variable is defined inside a function and can only be used within that function.

```
def saluta():  
    messaggio = "Ciao!" # local variable  
# print(messaggio) # error, messaggio is outside the global scope  
Saluta()
```

The variable `messaggio` exists only during the execution of the function `saluta`.
After the function ends, the variable is removed from memory.

Global variavle

- A global variable is defined outside all functions and can be used throughout the file, including inside functions (but with caution).

```
x = 10 # variabile globale

def modifica():
    x = 20 + x # change the value of x

modifica()
print(x) # Output: 10
```

Timelife of variables

- Local variables exist only during the execution of the function in which they are defined.
- Global variables exist as long as the program is running (that is, for the entire duration of the Python file).

Mechanism for accessing a variable inside a function

```
X = 10

def f():
    X = 20
    print(X) # usa la variabile locale X

f() # Output: 20
```

First, it looks inside the function, and if X is present—that is, if there is a local variable X inside f—it uses the local variable.

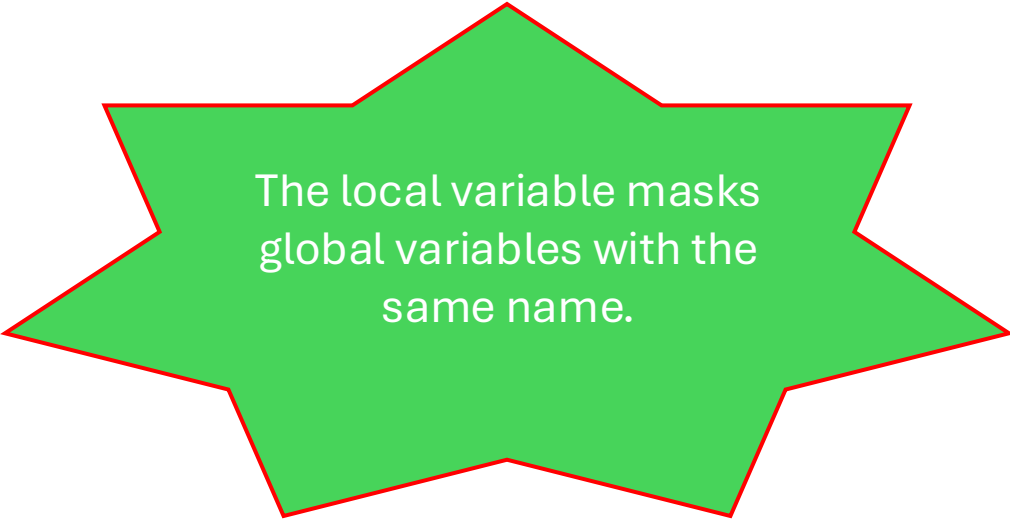
Meccanismo di accesso ad una variabile dentro una funzione

```
X = 10

def f():
    X = 20
    print(X) # globale X

f() # Output: 20
```

First, it looks inside the function, and if X is not present—that is, if there is NO local variable X inside f—it looks for the global variable X and uses the global variable X.



The local variable masks global variables with the same name.

Posso modificare da dentro una funzione una variabile globale

```
X = 10

def f():
    X = 20
    print(X) # usa la variabile locale X

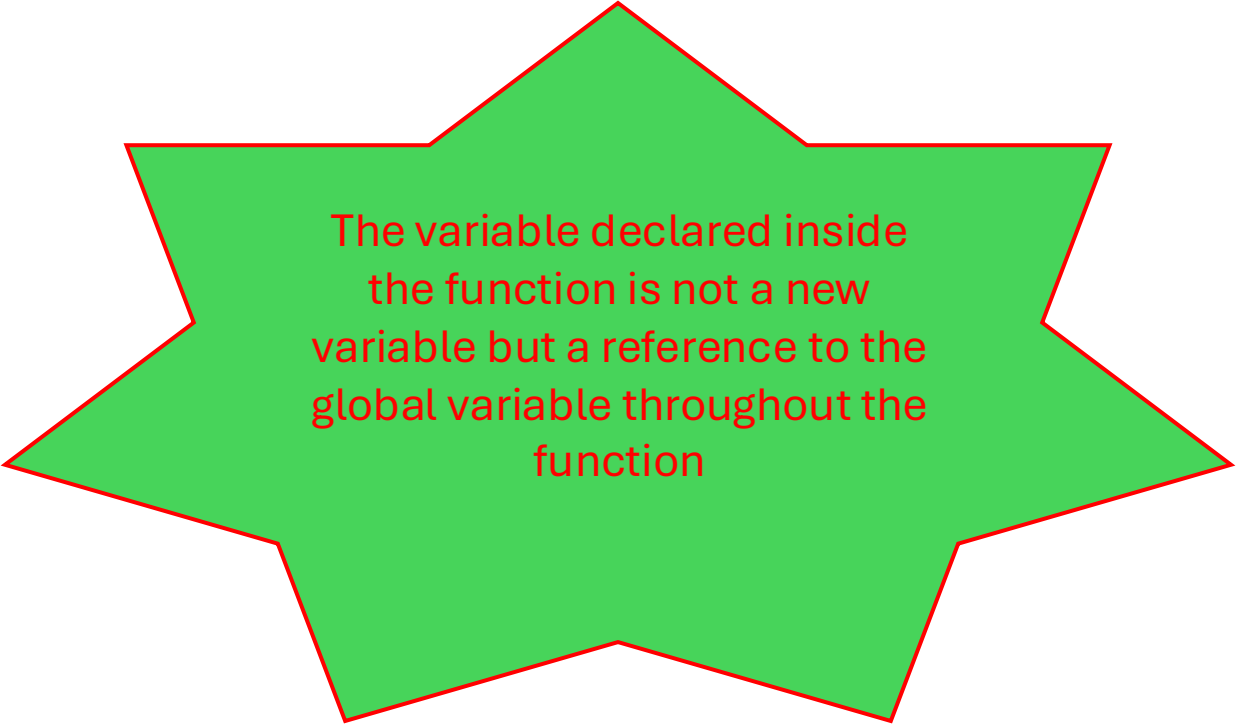
f() # Output: 20
print(x)
```

The global variable X is not modified; the variable to which we assign 20 is another variable. This second variable is local and is deleted when the function ends.

Il modificatore global

```
X = 10
def f():
    global X
    X = 20
    print(X)

print(X) #output 20
f()      #output 20
print(X) #output 20
```



The variable declared inside the function is not a new variable but a reference to the global variable throughout the function

Import Modules in Python

```
import <filename>
```

- The functions that we defined with import are not immediately imported into the present program. Only the name of the module is imported here.
- We may use the dot operator to use the functions using the module name.
- There are several standard modules for Python. The list can be seen using the help command.
- Similar to how we imported our module, a user-defined module, we can use an import statement to import other standard modules

Filename = example_module.py

```
1.# Python program to show how to create a module.  
2.# defining a function in the module to reuse it  
3.def square( number ):  
4.    """This function will square the number passed to it"""  
5.  
6.    result = number ** 2  
7.    return result
```

```
import example_module  
result = example_module.square( 4 )  
print( "By using the module square of number is: ", result )
```

Module search path

When a module is imported, Python does not know where it is located so it will look for the module in the following places, in order:

- Built-in modules.
- The directories listed in the `sys.path` variable.
- The `sys.path` variable is initialized from these locations:
 - The current directory.
 - `PYTHONPATH` (a list of directory names, with the same syntax as the shell variable `PATH`).
 - The installation-dependent default.

Package

A #package

A.B #submodule named B in a package named A

- When importing the package, Python searches through the directories on `sys.path` looking for the package subdirectory.
- `__init__.py` files are required to make Python treat directories containing the file as packages.
- This prevents directories with a common name, such as `string`, unintentionally hiding valid modules that occur later on the module search path.
- In the simplest case, `__init__.py` can just be an empty file, but it can also execute initialization code for the package or set the `__all__` variable, described later.

```
print("Inizio A")
```

```
import b
```

```
print("Fine A")
```

A.py

run A

Inizio A

Inizio B

Fine B

Fine A

```
print("Inizio B")
```

```
import a
```

```
print("Fine B")
```

B.py

run B

Inizio B

Inizio A

Fine A

Fine B

Multiple imports have no effect.

```
print("Inizio A")
```

```
import b
```

```
print("Fine A")
```

```
Import b
```

a.py

run A

Inizio A

Inizio B

Fine B

Fine A

```
print("Inizio B")
```

```
import a
```

```
print("Fine B")
```

```
Import a
```

b.py

Run B

Inizio B

Inizio A

Fine A

Fine B

Uso di componenti di un altro script

```
import b  
print("Fine A")  
  
print("Fine A")
```

a.py

Esecuzione A

Inizio A
Fine A

```
def f():  
    print('ciao')
```

b.py

Esecuzione B



```
import b  
print("Fine A")
```

```
b.f()
```

```
print("Fine A")
```

a.py

run A

Inizio A
Ciao
Fine A

```
def f():  
    print('ciao')
```

b.py

run B



Importing and Renaming

```
# Python program to show how to import a module and rename it  
# We will import the math module and give a different name to it
```

```
import math as mt  
print( "The value of euler's number is", mt.e )
```

from...import Statemen

```
# Python program to show how to import specific objects from a module  
# We will import euler's number from the math module using the from keyword
```

```
from math import e  
print( "The value of euler's number is", e )
```

```
from math import e, tau  
print( "The value of tau constant is: ", tau )  
print( "The value of the euler's number is: ", e )
```

Import all Names - From import * Statement

```
from name_of_module import *
```

```
# importing the complete math module using *
```

```
from math import *
```

```
# accessing functions of math module without using the dot operator
```

```
print( "Calculating square root: ", sqrt(25) )
```

```
print( "Calculating tangent of an angle: ", tan(pi/6) ) # here pi is also imported from the math module
```

__main__ cosa e'?

```
if __name__ == "__main__":  
    # This block runs only when the script is executed directly  
    print("Running as the main program")
```

- In Python, `__main__` is a special name used to indicate the **entry point of a program**.

Here's what it means:

- When you run a Python script directly (e.g., `python myscript.py`), the interpreter sets the variable `__name__` to `"__main__"`.
- If the file is **imported as a module** into another script, `__name__` will not be `"__main__"`; it will be the module's name instead. This prevents certain code from running unintentionally when importing.
- In short: `__main__` is used to distinguish between **running a file directly** and **importing it as a module**.

Uso di componenti di un altro script

```
import b
```

```
print('Sono A')
```

a.py

Esecuzione A

Sono B
sono A

```
import a
```

```
Print ('Sono B')
```

b.py

Esecuzione B

Sono C
sono B

Uso di componenti di un altro script

```
import b  
print("sono A")  
if name==__main():  
    print("sto eseguendo A")
```

a.py

Esecuzione A

sono B
Sono A
sto eseguendo B

```
print("sono B")  
if name==__main():  
    print("sto eseguendo B")
```

b.py

Esecuzione B

Sono B
sto eseguendo B

Import circolari → evitarli

```
import b
print('Sono A')
if name==__main():
    print("sto eseguendo A")
```

a.py

Esecuzione A

```
sono A
Sono B
Sono A
sto eseguendo B
```

```
Import a
print('sono B')
if name==__main():
    print("sto eseguendo B")
```

b.py

Esecuzione B

```
Sono B
Sono A
Sono B
sto eseguendo B
```

Quando due moduli si importano a vicenda, Python può eseguire parzialmente entrambi e quindi stampare due volte alcune righe di codice globale (cioè quelle *fuori* da funzioni)

Parameter typing

- In Python, **parameter passing does not enforce types by default** because Python is a dynamically typed language. This means you can pass any type of argument to a function without declaring its type explicitly.
- However, starting from **Python 3.5**, you can **indicate the expected types** using **type hints** (also called type annotations). These hints are optional and are mainly for readability and tools like linters or type checkers (e.g., mypy), not for runtime enforcement.

```
def add(a: int, b: int) -> int:  
    return a + b
```

```
result = add(3, 4) # Works fine  
print(result)
```

```
result = add(3.4, 2.3)  
print(result)
```

7

5.6999999999999999

`a: int` and `b: int` indicate that the function expects integers.

`-> int` indicates that the function returns an integer.

Important: Python does **not** enforce these types at runtime. They are just hints.

Enforce type checking

- To **enforce type checking at runtime in Python**, you have a few options because Python's built-in type hints are not enforced automatically.
- The main approaches are:
 1. Manual Type Checking with `isinstance()`
 2. Using `typeguard` Library
 3. Using `pydantic` for Data Validation

Manual Type Checking with isinstance()

```
def add(a: int, b: int) -> int:
    if not isinstance(a, int) or not isinstance(b, int):
        raise TypeError("Both arguments must be integers")
    return a + b

print(add(3, 4))          # Works
print(add("3", "4"))     # Raises TypeError, No works
```

Manual Type Checking with isinstance()

How it works: You explicitly check each argument inside the function using isinstance().

Pros:

- No external dependencies.
- Full control over error messages and logic.

Cons:

- Requires writing repetitive code for every parameter.
- Harder to maintain for large functions or complex types.

Use case: Simple functions with a few parameters.

Using typeguard Library

```
pip install typeguard

from typeguard import typechecked

@typechecked
def add(a: int, b: int) -> int:
    return a + b

print(add(3, 4))          # Works
print(add("3", "4"))     # Raises TypeError
```

Using typeguard Library

How it works: Decorate your function with `@typechecked`. It automatically enforces type hints at runtime.

Pros:

- Very easy to use—just add a decorator.
- Works with type hints (including complex types like `List[int]`).

Cons:

- Adds a runtime overhead.
- Requires installing an external library.

Use case: When you want automatic enforcement without writing manual checks.

Using pydantic for Data Validation

```
from pydantic import validate_arguments

@validate_arguments
def add(a: int, b: int) -> int:
    return a + b

print(add(3, 4))          # Works
print(add("3", "4"))     # Raises ValidationError
```


Using pydantic for Data Validation

How it works: Uses `@validate_arguments` or Pydantic models to validate and convert data.

Pros:

- Powerful validation and type conversion (e.g., converts "3" to int).
- Great for APIs, data models, and complex validation rules.

Cons:

- Heavier dependency compared to typeguard.
- Slightly more complex setup.

Use case: When you need strict validation and data parsing (e.g., FastAPI, data pipelines).

```
from pydantic import validate_arguments

@validate_arguments
def add(a: int, b: int) -> int:
    return a + b

print(add(3, 4))          # Works
print(add("3", "4"))     # Converts and works (returns 7)
print(add("abc", "4"))   # Raises ValidationError
```

Summary Table

Feature	isinstance()	typeguard	pydantic
External lib	No	Yes	Yes
Ease of use	Low	High	Medium
Supports complex types	Manual	Yes	Yes
Validation + Conversion	No	No	Yes
Performance	Fast	Moderate	Moderate